Décembre 2025

Projet d'Algorithmique Avancé: Huffman Dynamique

Yvan PARENT
M1 Informatique STL
yvan.parent@etu.sorbonne-universite.fr

Laura LY
M1 Informatique STL
laura.ly@etu.sorbonne-universite.fr

Table des matières

1. Introduction : Démonstrations **1**
1.1. Question 1 1
1.2. Question 2 3
2. Choix du langage et des structures de données **4**
2.1. Choix du langage 4
2.2. Makefile 4
2.3. Question 5 : Choix des structures de données 4
2.4. Complexités 5
2.5. Architecture du logiciel 6
2.6. Tests 7
2.7. Étude expérimentale 7
2.8. Bonus : Régler le problème de terminaison des décompressions 8
2.9. Bonus : Visualiseur d'arbres 9
3. Usage d'IA **10**

1. Introduction : Démonstrations

Cette première partie est consacrée à répondre aux premières questions de démonstrations posées dans l'énoncé.

1.1. Question 1

Soit A un AHA et x_n et x_m (avec $m > n$ par numérotation GDBH) tel que x_m est un des ancêtres de x_n.

Par l'absurde, supposons que l'algorithme de modification a échangé le nœud x_n avec x_m. Cela signifie qu'à un moment, la fonction `finBloc(A, x_n)` a renvoyé x_m, d'où par définition on a eu :

$$W(x_n) = W(x_{n+1}) = ... = W(x_m) \tag{i}$$

et

$$W(x_m) < W(x_{m+1}) \tag{ii}$$

Cas 1 : x_m n'est pas le parent direct de x_n

En particulier, (i) implique que tous les nœuds x_i internes ancêtres de x_n et descendants de x_m

ont le même poids que ces deux nœuds, et donc que chacun de ces x_i (ainsi que x_m), a un de ses enfants qui est une feuille dont le poids est nul (i.e. $W(x_i) = 0$).

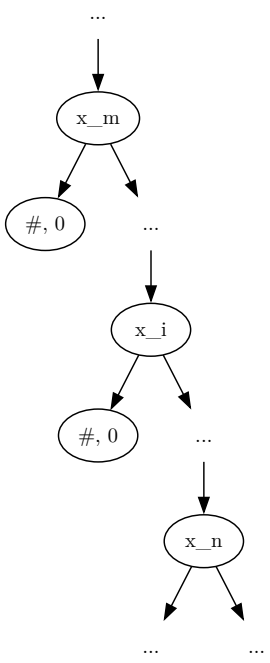

Fig. 1. – Schéma de la configuration des nœuds

Ce qui est impossible puisqu'il ne peut y avoir qu'un seul nœud de poids 0, et il doit se trouver plus bas et plus à gauche de x_n par définition d'un AHA. (Ici, les feuilles de poids 0 sont au-dessus de x_n, avec $W(x_n) \neq 0$ car sinon on aurait $W(x_n) = W(x_m) = 0$ donc plusieurs nœuds de poids nuls ce qui est également impossible.)

Cas 2 : x_m est parent immédiat de x_n

Alors comme x_m et x_n ont le même poids, le deuxième enfant de x_m est forcément la feuille dont le poids est nul, par définition. D'après l'algorithme de modification, on tombe exactement dans le cas du `else`:

```
def Modification(H, s):
    """ AHA * Symbole -> AHA
        Renvoie l'arbre de Huffman avec s incrémenté"""

    if H == #:
        H devient arbre avec un noeud interne (pds 1)
            avec enfant gauche #
            et enfant droit s (pds 1)
        return H

    elif s not in H:
        Q = parent(feuille #)
        Remplacer noeud(#) par noeud (pds 1)
            avec enfant gauche #
            et enfant droit s (pds 1)

    else:
        Q = feuille(s)
        if enfants(parent(Q)) == {Q, #} and parent(Q) == finBloc(H,Q):
            Ajouter 1 au poids de Q
            Q = parent(Q)

    return Traitement(H, Q)
```

Fig. 2. – Pseudo-code de `Modification` dans le cours 1

Ces deux nœuds ne sont alors pas échangé et l'appel à `Traitement` fait possiblement des échanges qui retombent dans le **Cas 1**.

■

1.2. Question 2

Pour tout $n \geq 1$, soit la propriété P(n) suivante :
« Si H est un AHA construit sur un alphabet de n symboles, alors l'ajout ou l'incrémentation d'une nouvelle lettre provoque au plus $n - 1$ échanges dans H. »
Afin de montrer que cette dernière est vraie pour tout n positif, on procède par récurrence.

- On prend notre cas de base avec H un AHA construit sur un alphabet de n = *1* lettre. La propriété $P(1)$ implique que l'alphabet possède un unique élément {a}, et qu'il n'est jamais nécessaire de le modifier suite à la lecture d'une nouvelle lettre "a" (il y a au plus n - *1* = *1* - *1* = 0 échanges d'après $P(1)$). On sépare alors 2 cas différents afin de prouver cela :
  1. L'arbre H est encore vide : lorsque l'on introduit pour la première fois la lettre "a" dedans, aucun ajustement n'est nécessaire puisqu'il suffit d'ajouter deux enfants ("#" et "a") au nœud racine de H.
  2. On a déjà lu "a" auparavant, et "a" possède un poids P quelconque : lorsque l'on ajoute une nouvelle fois la lettre "a", on incrémente donc le poids de "a" de 1, ainsi que le poids de son parent de la même façon. Son parent étant le nœud racine, aucun autre élément n'a besoin d'être incrémenté. De plus, H suit toujours l'ordre GDBH conventionnel puisque l'arbre ne possède que deux éléments "#" et "a", ayant respectivement comme poids 0 et (p+1).

  Ainsi, dans les deux cas, on a bien aucun changement qui ne se produit lors de l'ajout d'une nouvelle lettre. On a bien prouver que la propriété P fonctionne pour le cas n = *1*.

- Soit un alphabet de taille k, et la propriété $P(k)$ vérifiée. On sait donc d'après $P(k)$ qu'il y a au maximum k - *1* échanges réalisés lors d'une modification de H. On veut prouver par récurrence que si $P(k)$ est vraie, on a également $P(k+1)$ qui est vraie :

  Soit T_0 un AHA construit à partir d'un alphabet de taille k vérifiant $P(k)$ et T' l'arbre obtenu à partir de T_0 après que l'ajout d'une lettre $l_i \in \{l_1, l_2, ..., l_{k-1}, l_k\}$ déclenche le maximum de k - *1* échanges depuis T_0. On pose P_{T_0} le poids de T_0 à sa racine, et $(P_{T_0} + 1)$ le poids de $P_{T'}$ à sa racine. Enfin, on défini la lettre l_{k+1}, non incluse de l'alphabet de T_0.

  On pose $(T_0)'$ l'arbre construit sur un alphabet de taille k - *1* ayant respectivement comme fils gauche et fils droit de son nœud racine l'arbre T_0 de poids P_{T_0} et la feuille l_{k+1} de poids P_{T_0}. Le poids total de $(T_0)'$ est donc de $2\left(P_{T_0}\right)$.

  Le principe de démonstration réside dans le fait de montrer que l'action de passage de T_0 à T' produit $k - 1 + 1 = k$ échanges lorsque T_0 a une lettre en plus. Ici donc quand on ajoute l_i à $(T_0)'$ à l'instant dans lequel il est :
  En ajoutant l_i à $(T_0)'$, on obtient par Hypothèse de Récurrence que son fils gauche a un nouveau poids total $(P_{T_0} + 1)$ après avoir réalisé k - *1* échanges.
  L'arbre ne suit alors pas encore la règle GDBH puisque le fils gauche de son nœud racine a un poids strictement supérieur à celui de son fils droit $((P_{T_0} + 1) > (P_{T_0}))$. Il est donc nécessaire de réaliser un dernier échange entre les deux fils afin de valider selon l'ordre GDBH le nouvel

arbre résultant de l'ajout de l_i, ce qui nous fait bien un total de $k - 1 + 1 = k$ échanges réalisés.

Ainsi, on a bien prouvé qu'il était possible de réaliser jusqu'à k échanges lors d'une modification d'un arbre d'un alphabet de (k+1) lettres, satisfaisant donc la propriété $P(k+1)$.

On a prouvé que la propriété P est valide pour $n = 1$, et que pour tout k rendant $P(k)$ vraie, on peut en déduire que $P(k+1)$ est également vraie. Par récurrence, on en déduit que P est vraie pour tout $k > 0$.

■

2. Choix du langage et des structures de données

2.1. Choix du langage

Nous avons d'abord rassemblé les langages avec lesquels nous étions familier: en particulier, nous pensions à Python, C et OCaml.
Nous avons écarté Python car trop haut niveau et les complexités de ses structures et fonctions sont trop floues, ce qui cacherait la complexité réelle qui serait probablement pire que celle qu'on évaluerait et qu'on souhaiterait avoir.
Nous avons également dû nous séparer de C, bien que algorithmiquement idéal car très proche de la machine et meilleure gestion de la complexité, car nous ne nous sentions pas assez à l'aise avec ce langage pour pouvoir réaliser le projet avec, et qu'il est plus facile de réaliser des erreurs non apparentes directement avec C.
Nous nous sommes donc résolu à utiliser OCaml, un langage avec lequel nous sommes tous les deux bien à l'aise, et qui reste plus lisible que C. Nous pouvons tout de même nous réjouir de ce choix, car bien qu'il n'est pas aussi bas niveau que C, la performance reste relativement bonne, notamment grâce à la compilation native via `ocamlopt`.

2.2. Makefile

Le makefile permet de lancer trois commandes principales :

- `make test` lançant les tests.
- `make analyse` lançant une analyse des performances avec les différentes études proposées.
- `make clean` pour clean le projet des fichiers de compilation

2.3. Question 5 : Choix des structures de données

Nous avons d'abord fait le choix de créer une structure pour la représentation des caractères dans l'AHA. Notamment,pour le caractère vide on ne pouvait pas utiliser le caractère # comme on le faisait dans le cours, car on rencontrerait une ambiguïté si le texte à compresser contenait un #. Nous avons donc le type `chara` :

```
type chara = EmptyChar | Char of Uchar.t
```

Pour la structure de l'AHA, nous avons créé des boites `btree`, représentant les nœuds et contenant deux champs mutables: `content` et `parent`. Chaque nœud a un `parent` sauf la racine, d'où le type `option`. Le contenu (type `btree_`) est soit celui d'une feuille, donc un `chara` et son nombre d'apparition, soit celui d'un nœud interne, i.e. son poids et l'accès à ses deux enfants.

```
type btree_ = Leaf of chara * int | Node of btree * int * btree
and btree = {mutable content : btree_; mutable parent : btree option}
```

De plus, afin de faciliter l'accès aux feuilles, notamment pour obtenir leur code ou incrémenter leur poids, nous disposons d'un dictionnaire (`btreeTable`) qui associe à un `chara` la feuille correspondante dans l'AHA.

```
module CharaKey : sig
  type t = chara
  val compare : t -> t -> int
end
module CharaMap : Map.S with type key = CharaKey.t
type btreeTable = btree CharaMap.t
```

2.4. Complexités

En ce qui concerne les différentes complexités associées à cette structure de données, voici une analyse de chacune des primitives :

[**vals_per_depth**] : Avec **n** le nombre de nœuds, la fonction fait un parcours de l'arbre en entier une unique fois, on a donc **O(n)** sur cette fonction.

[**node_per_depth**] : Similaire à `[vals_per_depth]` en tout point, **O(n)** également.

[**is_gdbh**] : La fonction commence par appeler `[vals_per_depth]`, on a donc un **O(n)** pour le moment. Ensuite, on parcours simplement chacune des listes obtenues, on a donc pas de complexité augmentée. Cette fonction est aussi **O(n)**.

[**insert**] : On a 2 cas différents : Si le caractère est ancien, on incrémente simplement le poids de la feuille de 1 (accessible via la table) : Complexité en **O(1)** | Sinon, si le caractère est nouveau, on accède via la table au #, et on le remplace par la nouvelle feuille : Complexité en **O(1)** car tout est accessible via la table.

[**mem**] : Complexité en **O(1)** puisque cela correspond simplement à rechercher un élément dans la table.

[**switch**] : Complexité en **O(1)** car le switch ne fait qu'accéder à des nœuds de manière directe par la table, met à jour les contenus, et fait au maximum deux Map.add par la suite.

[**finBloc**] : On parcours au pire des cas chacun des nœuds de l'arbre si le poids que l'on cherche se trouve tout en bas de l'arbre : on a donc un **O(n)**.

[**parent**] : Complexité en **O(1)** car le parent d'un nœud est enregistré dans le nœud lui même.

[**chemin**] : Complexité en fonction de la hauteur de l'arbre (car on accède aux parents récursivement) : on obtient **O(h)** avec h la hauteur de l'arbre, soit **O(n)** dans le pire des cas.

[**is_incrementable**] : On appelle `[node_per_depth]`, ce qui nous donne d'emblée un **O(n)**, puis on fait un calcul pour chaque profondeur (dans le pire cas où on regarde l'incrémentation depuis la plus basse profondeur) : on a donc un **O(n*h)**.
Nous avons essayé de rendre cette fonction plus efficace, notamment en renvoyant une Hashtable qui, prenant un nœud en clé, renvoie le nœud suivant. Théoriquement, cela permettrait d'éviter de parcourir des listes inutilement par la suite, mais nous avons malheureusement

rencontré plusieurs problèmes sur la nouvelle implémentation (principalement lié au fait qu'il est compliqué de mettre un btree valide en clé qui le reste toujours par la suite). Nous sommes donc resté sur cette version dont nous sommes sûr qu'elle fonctionne correctement.

[**traitement**] : Durant un appel récursif de cette fonction, on appelle `[is_incrementable]` en **O(h)**, `[increment]` en **O(h)**, `[update_weights]` en temps constant car utilisé sur #, `[finBloc]` en **O(n)**, `[switch]` en **O(1)** et `[increment_until]` en **O(h)**. De plus, on a au pire h appels récursifs car la profondeur est réduite de 1 à chaque tour de fonction. On a donc une complexité en moyenne de **O((n+h)*h)**.

[**modification**] : Appel de traitement, on a donc la même complexité dans le pire cas en **O((n+h)*h)**.

[**code**] : Complexité en **O(log(n))** car on parcours récursivement les parents à partir d'une feuille jusqu'à la racine afin d'obtenir le code, donc au max h visites, h étant la hauteur de l'arbre.

[**compression**]: Pour chaque caractère du texte de taille t, on effectue comme opérations sur l'arbre les fonctions `[code]` et `[modification]`, on obtient donc comme complexité **O((n+h)*h*h)**

[**decompression**]: Ici c'est plus compliqué puisqu'on ne calcule pas "pour chaque caractère dans le texte", mais sur chaque bit du texte. Cependant on va relire plusieurs fois une bonne partie des bits, mais jamais plus de 2 ou 3 fois, donc au pire on a un facteur ajouté constant à la taille de texte, ce qui ne change pas la complexité : on note t la taille du texte en bit également. A chaque étape de la compression, on utilise `[modification]` et la fonction `[find_next_code]` qui est en **O(h)** avec h la hauteur puisqu'elle parcours tout l'arbre depuis la racine jusqu'à une feuille (potentiellement tout en bas). On obtient alors une complexité en **O((n+h)*h*h)**

2.5. Architecture du logiciel

Nous avons donné une importance particulière à l'architecture, la présentation et la forme de notre logiciel. On trouve dans le répertoire principal le makefile, le README, deux fichiers bash pour lancer la compression ou la décompression, ainsi que les différents fichiers ocaml nécessaire pour l'algorithme : ces fichiers sont compression.ml, compression.mli, decompression.ml, decompression.mli, primitive.ml, `primitive.mli`, `utils.ml` et `utils.mli`. Chacun des fichiers `.mli` contient une description détaillée de ce que les fonctions font, et chaque fichier `.ml` à un fichier `.mli` détaillé associé. De plus, des commentaires sont présents dans les codes des fonctions lorsque cela a été jugé nécessaire.
Dans ce même répertoire, on trouve un dossier `/test_files/` regroupant toute l'infrastructure nécessaire aux tests et aux différentes analyses réalisées : cela est explicité dans les parties suivantes respectives.
Le principal avantage de notre environnement est qu'il est très facilement modulable, dans le sens où il est très aisé de rajouter de nouveaux tests et de nouvelles études, sans que cela soit brouillon, puisque les fichiers sont pensés pour accueillir un nombre indéfini de tâches (par exemple, pour rajouter un nouveau livre à compresser dans l'étude construite à partir du projet de Gutenberg, il suffit de rajouter ce fichier texte au dossier `/Gutemberg/`, puis de rajouter son nom dans le fichier `etude_gutemberg.ml` dans la liste **files_to_use** : il est alors rajouté automatiquement à l'étude).

2.6. Tests

Toutes les fonctions des différents fichiers ont été soigneusement testé, ces tests sont regroupés leur appartenance à des fichiers communs. En effet, dans le dossier `/test_files/`, on peut trouver `test_compression.ml`, `test_decompression.ml`, `test_primitive.ml` et `test_utils.ml`, testant les fichiers de code correspondants à chacun de leur nom. Ensuite, le fichier `test.ml` se charge quand à lui de lancer les différents fichiers de test. Pour fonctionner, ces tests utilisent l'espace du dossier `/ressources/`, ainsi que le sous-dossier `/Blaise_Pascal/` pour le test de compression et décompression.

2.7. Étude expérimentale

Afin de répondre à la création de l'étude expérimentale, différents fichiers ont été mis en place pour chacun des types d'étude.

Dans le dossier `/test_files/`, nous avons ajouté trois fichiers d'étude : `etude_random.ml`, `etude_autre_language.ml`, et `etude_gutemberg.ml`. Ces fichiers permettent de lancer des compressions et des décompressions sur les différents fichiers présents dans les dossiers respectifs à l'intérieur de `/ressources/`.

Le dossier `/Gutemberg/` possède plusieurs gros textes provenant de livres de la littérature française à tester.

`/Random/` contient quelques fichier totalement aléatoires de tailles variables, des exemples de fichiers ayant peu voir qu'un seul caractère, et par opposition des exemples sur des textes n'ayant quasiment, voir que des caractères uniques.

Dans `/Language/`, nous avons voulu testé les algorithmes sur un fichier `.json`, ainsi qu'un fichier `.py`.

Ensuite, comme pour l'infrastructure de test, le fichier `analyse.ml` se charge de lancer les différentes études.

Les résultats sont affichés dans le terminal après avoir lancé la commande `make analyse`, voici une retranscription de ces résultats :

Fichiers	**Nombre d'octets initial**	**Nombre d'octets compressés**	**Nombre d'octets décompressés**	**Taux de compression**	**Taux de décompression**
`Curiosités judiciaires et historiques du moyen âge. Procès contre les animaux.txt`	101060	59111	98719	0.58491	1.67006
`Dernières lettres d'un bon jeune homme à sa cousine Madeleine.txt`	483163	273478	471924	0.56602	1.72564
`Pierre de Villerglé.txt`	279829	159469	272894	0.56988	1.71127
`randomText1.txt`	4003	3107	4001	0.77617	1.28774
`randomText2.txt`	12162	9272	12160	0.76237	1.31148
`all_a.txt`	19998	2504	19999	0.12521	7.98682
`randomText3.txt`	300003	226645	300001	0.75548	1.32366
`worst_case.txt`	262	390	144	1.48855	0.36923
`all_a_b.txt`	225504	42417	225505	0.18810	5.31638
`randomText4.txt`	1643546	880543	1643547	0.53576	1.86652
`fibo.py`	98	78	100	0.79592	1.28205

Fichiers	**Nombre d'octets initial**	**Nombre d'octets compressés**	**Nombre d'octets décompressés**	**Taux de compression**	**Taux de décompression**
`database.json`	43713	25033	43440	0.57267	1.73531

On peut ainsi remarquer qu'on obtient expérimentalement le comportement attendu pour la compression. Dans le cas extrême où le texte est assez long avec très peu de caractères différents comme dans `all_a.txt` (qui ne contient que des a) ou dans `all_a_b.txt` (qui ne contient que des a et des b), le taux de compression est inférieur à 0.2 et donc faible ce qui indique une très bonne compression puisque la taille du texte compressé est significativement inférieure à celle du texte original. À l'inverse, dans le cas où la variété de caractères est large et relativement court comme dans `worst_case.txt`, la compression se révèle assez mauvaise puisque qu'elle allonge la taille du texte initial. Pour des cas plus proche de ce qu'on peut retrouver usuellement, comme des textes (comme `Pierre de Villerglé.txt`) ou du code (`database.json`), la compression est également satisfaisante, puisqu'elle divise presque de moitié la taille du texte initial (le taux de compression est proche de 0.5).

Pour ce qui est de la décompression, on n'arrive pas à obtenir le même nombre d'octets que le texte initial, même si on arrive dans certains cas à s'en approcher, comme pour `fibo.py` ou `randomText3.txt` (qui n'ont que 2 octets de différence). On pourrait potentiellement expliquer cela par l'absence du caractère `\r` (rattaché à celle des sauts de ligne) à la décompression, bien que cette hypothèse pourrait être réfutée par la présence nombreuse de saut de ligne dans des codes comme `fibo.py`.

2.8. Bonus : Régler le problème de terminaison des décompressions

Actuellement, le compresseur (et par extension le décompresseur) a un problème dans sa conception même qui fait que le résultat n'est pas toujours parfait lors d'une décompression. En effet, illustrons ce problème en prenant l'exemple du texte `Blaise_Pascal.txt`, en utilisant le compresseur **avant** la modification apportée sur la lecture des **'\r'** de la version finale :

En suivant l'algorithme de compression du sujet, lorsqu'il ne reste plus aucun bits à rajouter, on rempli les derniers quelques bits restants, et on rempli le dernier octet de bits **0** de façon à obtenir un octet entier. Sur notre exemple, la compression se termine sur un bit **0**, donc l'octet associé est rempli de sept bits **0** afin de compléter ce dernier octet. Cependant, il s'avère également qu'à ce moment de la compression, le code du caractère **'a'** dans l'arbre est **0000**. De cette situation, on obtient que le code généré par la compression du texte Blaise_Pascal.txt est totalement identique à celui obtenu en compressant ce même fichier dont on ajoute la lettre **'a'** à la fin.

Cela pose également problème lors de la décompression, puisque par défaut le décompresseur va chercher à décompresser le plus de caractères possible, engendrant donc dans tous les cas l'apparition de la lettre **'a'** à la fin du fichier décompressé.

Mais alors, que peut-on faire ? Nous avons trouvé deux façon de permettre de régler ce problème, sans pour autant changer la logique des algorithmes, et en ajoutant le moins de bits possible en plus :

- L'une des deux solutions serait de rajouter au tout début du fichier une information concernant la fin du fichier (par exemple juste après es 3 premiers octets BOM). En particulier, on pourrait décrire le tout dernier octet sur l'importance de chacun des bits, avec un bit **1** si le bit est utilisé, et un **0** autrement. Par exemple, pour retourner sur notre exemple de Blaise_Pascal.txt, on aurait comme quatrième octet la suite **1000 0000** si on ne veut pas lire le **'a'**, et **1111 1000** si le caractère est important.

- La deuxième solution, moins "propre", ne modifie que la fin du fichier compressé. Pour le comprendre, il faut d'abord se rappeler que lors des étapes de l'algorithme de décompression, on essaie d'abord de lire le code de #, puis si on a pu le lire, on lit ensuite le code d'un caractère UTF-8, donc au moins 8 bits nécessaires par la suite. Ici , nous proposons de rajouter à la fin de la compression le code de # une dernière fois, puis de compléter le reste par des bits **0** comme c'est déjà le cas, c'est-à-dire qu'on ne rajoutera au maximum que sept bits **0**. Lors de la décompression, la lecture des sept derniers bits (toujours au maximum) ne permet pas de définir un nouveau caractère UTF-8, signant alors l'arrêt de la décompression. Dans notre exemple de Blaise_Pascal.txt, supposons que **1111 1111 11** est le code de # dans l'arbre, le code inscrit à la fin de la compression dans le cas où l'on ne veut pas de **'a'** serait **0111 1111 1110 0000**. Dans l'autre cas, on aurait alors **0000 0111 1111 1110**, ces deux paires d'octets étant bien différents.

2.9. Bonus : Visualiseur d'arbres

Nous avons implémenté une fonction permettant d'obtenir la représentation d'un arbre en fichier `.dot` compatible avec GraphViz. Cela est fait en parcourant l'arbre récursivement : chaque nœud est associé à un identifiant unique grâce à `Hashtbl.hash`, puis sa représentation est ajouté dans le fichier en fonction de si c'est un nœud ou une feuille (nous avons fait le choix de ne pas donner de forme, ni de couleur aux nœuds internes, et de faire des ellipses vertes pour les feuilles), ensuite nous appelons récursivement la fonction pour créer les nœuds enfants, et enfin nous créons les liens entre parents et enfants.

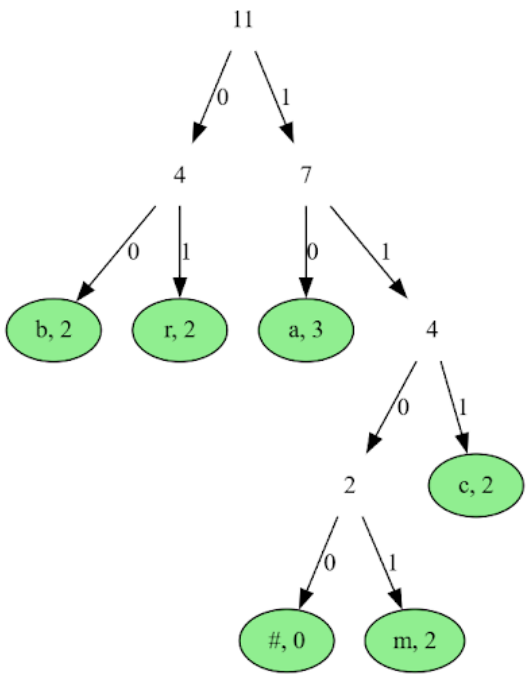

Fig. 3. – AHA après compression de CARAMBARBCM, par notre algorithme de visualisation

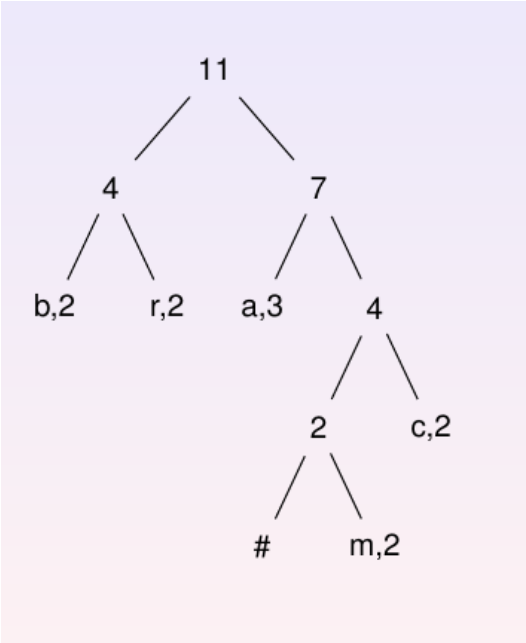

Fig. 4. – AHA après compression de CARAMBARBCM, comme visualisé dans le cours

Cela nous a notamment permis de mieux visualiser et comparer les premières tentatives de compression sur les exemples du cours et des TDs, ce qui a facilité l'identification de bugs.

3. Usage d'IA

Nous avons dû utiliser de l'intelligence artificielle générative pour construire une partie du *Makefile*. En effet, nous ne sommes pas très à l'aise avec les *Makefiles*, comme OCaml est plus souvent utilisé avec *Dune* et que nous avions réalisé peu de projets (en langages compilés) qui nécessitaient l'utilisation de *Makefiles* dans le passé. En rétrospective, le fonctionnement d'un *Makefile* est relativement simple et nous aurions pu prendre le temps d'en faire un manuellement, mais nous avions eu déjà beaucoup de mal à faire fonctionner nos premières tentatives à causes des multiples dépendances et nous avions peur de perdre trop de temps pour quelque chose qui ne reflétaient pas particulièrement des compétences d'algorithmique.

En dehors de la génération de code directe, nous avons utiliser de l'IA pour obtenir plus d'explications sur le fonctionnement de UTF8 et notamment comment les décodeurs UTF8 fonctionnaient sur OCaml. Il y avait effectivement trop peu d'informations et de documentations sur les bibliothèques en question, et nous avions dû demander à une IA de nous générer un exemple similaire à l'utilisation que nous en avons fait:

```
let utf8_decoder (ic : in_channel) =
            let decoder = Uutf.decoder ~encoding:`UTF_8 (`Channel ic) in
            fun () ->
                match Uutf.decode decoder with
                | `Uchar u -> Some u
                | `End -> None
                | `Malformed _ -> None
                | _ -> None
        in
```

dans la compression. Nous avons d'abord cherché à comprendre cette section donnée, en l'associant avec les informations collectées avec nos recherches, avant de l'incorporer dans notre code. Malgré le fait que cette courte section ait été générée automatiquement, nous sommes en mesure de pouvoir l'expliquer dans sa globalité.